Ders 18 Genişletilmiş Notu: Dizilerde Temel İşlemler: Arama, Ekleme, Silme ve Bubble Sort

Ders 18 Genişletilmiş Notu: Dizilerde Temel İşlemler: Arama, Ekleme, Silme ve Bubble Sort

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dizi Üzerindeki Temel İşlemler
- 2. Çift Tırnak ile Tek Tırnak Arasındaki Fark
- 3. Sırasız Dizide Arama
- 4. Sıralı Dizide Arama
- 5. Diziye Eleman Ekleme
- 6. Diziden Eleman Silme
- 7. Güncelleme ve Günlük Hayattan Örnekler
- 8. Kaydırma Maliyetinden Kaçınmak
- 9. Sıralama Problemi ve Bubble Sort

1. Dizi Üzerindeki Temel İşlemler

Hoca derse, diziler ve veri setleri üzerindeki temel işlemleri (basic operations) konuşacaklarını söyleyerek başladı. Her problem bu temel işlemleri belli oranda içerir. Bu yüzden önce bu işlemlerin adını koymak gerekir. Hoca ilkini kendisi söyledi, gerisini öğrencilerin bulmasını istedi: Bir dizi üzerinde arama yapabiliriz, yani istediğimiz bir bilginin dizide olup olmadığına bakabiliriz.

Öğrencilerden gelen öneriler üzerinden liste şöyle tamamlandı:

- **Elemanların sırasını değiştirmek:** Yapılabilir, ama hoca önce daha temel bir işlem istedi.
- **Diziye bir atama yapmak:** Hoca bunu “ekleme” olarak yorumladı. Bu, temel işlemlerden biridir.
- **Eleman sayısını bulmak:** Yapılabilir, ama genelde dizinin boyutu zaten verilir. Bu yüzden hoca bunu temel işlem olarak almadı.
- **Eleman çıkarmak:** Bu da temel işlemlerden biridir, silme (delete).
- **İki diziyi birleştirmek:** Burada tek bir dizi üzerinde çalışıldığı için hoca bunu listeye almadı.
- **Herhangi bir elemanı doğrudan değiştirmek:** Bunun adı güncellemedir (update).

Böylece veri üzerinde yapılabilecek dört temel işlem ortaya çıktı:

- **Arama (search):** Bir bilginin veride olup olmadığına bakmak.
- **Ekleme (insert):** Veriye yeni bir bilgi eklemek.
- **Silme (delete):** Veriden bir bilgiyi silmek.
- **Güncelleme (update):** Veri üzerinde değişiklik yapmak.

2. Çift Tırnak ile Tek Tırnak Arasındaki Fark

Konuya geçmeden önce hoca, sohbet penceresinden gelen bir soruyu cevapladı.

Soru: Çift tırnaklı `"A"` ile tek tırnaklı `'A'` arasındaki fark nedir?

Cevap: Çift tırnak kullanıldığında `A` ve onu izleyen null karakteri vardır, yani birden fazla karakter söz konusudur. Tek tırnak kullanıldığında ise tek bir `A` sabiti vardır. Birincisi bir karakter dizisinde, ikincisi bir karakter değişkeninde saklanır.

3. Sırasız Dizide Arama

İlk işlem olarak arama (search operation) ele alındı. Hoca önce elimizde bir dizi olduğunu varsaydı ve iki soru sordu: Arama nasıl yapılır ve kaç farklı durum olabilir?

Bir öğrenci, döngü içinde `if` kullanarak aranan değeri bulunca döngüden çıkmayı önerdi. Hoca buradan iki durum çıkardı: aranan elemanı bulmak ve bulamamak.

Döngü seçimi de sorunun ne istediğine bağlıdır:

- Aranan eleman bir kez geçiyorsa ya da bir kez bulunması isteniyorsa `while` döngüsü kullanılır. Eleman bulununca döngüden çıkılır.
- Eleman birden fazla geçiyorsa ve her birinin yerini bulmak isteniyorsa `for` döngüsü kullanılır. Çünkü bütün diziye bakmak gerekir.

3.1 Akış Diyagramı

Hoca, daha hızlı ilerlemek için dizi okumayı `N, Dizi[N]` kısaltmasıyla yazdı. Bu kısaltma derste yalnız zaman kazanmak içindir. Hoca bu konuya ilerleyen bölümlerde tekrar değindi (bkz. 5.8).

```
1. Başla.
2. N ve Dizi[N] oku.
3. Aranan elemanı X olarak oku.
4. I = 1.
5. I <= N ve Dizi[I] ≠ X olduğu sürece:
     I = I + 1.
6. I <= N ise I yaz (eleman bulundu).
   Değilse "Eleman bulunamadı" yaz.
7. Bitir.
```

Döngü koşulu önce `I < N` diye yazıldı, hemen ardından `I <= N` olarak düzeltildi. Döngüden çıktıktan sonra ne yapılacağı için bir öğrenci `I = N` ya da `Dizi[I] = X` kontrolünü önerdi, hoca ise `I <= N` kontrolünü yazdı.

> **Ek bilgi (derste anlatılmadı):** Hoca döngüyü izlerken `I <= N` yanlış olduğunda ikinci koşula bakmadan çıktı. Akış diyagramındaki "ve" bağlacı büyük olasılıkla C'deki `&&` gibi soldan sağa değerlendirilir. Birinci koşul yanlışsa ikincisi hesaplanmaz, yani `Dizi[N+1]` okunmaz. Bu sıra değişirse dizinin dışına erişilir, bu yüzden sınır kontrolü büyük olasılıkla solda yazılmalıdır.

3.2 Eleman Dizide Yoksa

Bir öğrenci algoritmanın yanlış çıktı verdiğini düşündü.

Soru: Dizinin sonuna geldiğimizde $I = N$ oluyor. Aşağıdaki `if` içinde de $I = N$ olduğu için bulunamamış olması gerekmiyor mu?

Cevap: Hoca 7 elemanlı dizide ($N = 7$) 9'u arayarak birlikte izledi. I sırayla 1, 2, ..., 7 olur. $I = 7$ için $7 \leq 7$ doğrudur ve `Dizi[7]` 9'a eşit değildir, yani döngüden çıkılmaz. Döngü içinde $I = 8$ olur. Şimdi $8 \leq 7$ yanlıştır ve döngüden çıkılır. Aşağıda yine $8 \leq 7$ sorulur, cevap hayırdır ve "Eleman bulunamadı" yazılır. Algoritma doğrudur.

3.3 Eleman Son Gözdeyse

Sohbetten gelen soru şuydu: Eleman son gözdeyse ne olur? Hoca bu kez 6'yı aradı, 6 dizinin son elemanıydı. $I = 7$ için $7 \leq 7$ doğrudur, ama `Dizi[7]` 6'ya eşit olduğu için döngüden çıkılır. Aşağıda $7 \leq 7$ doğru olduğu için ekrana 7 yazılır ve program biter.

4. Sıralı Dizide Arama

Hoca ardından verilerin sıralı olduğu bir diziyi sordu: Ne fark eder, aynı algoritma kullanılabilir mi? Bir öğrenci hem aynı algoritmanın kullanılabileceğini hem de daha iyisinin yazılabileceğini söyledi. Hoca iki tespiti de onayladı.

Daha iyi çözüm için üç öneri geldi:

- **İlk elemanı aranandan çıkarıp sonuca göre gözü bulmak:** Bunu öneren öğrenci, dizi birer birer artmadığı için çözümün tutmadığını kendisi fark etti.
- **Aramaya ortadan başlamak:** Eleman ortadakinden büyükse ileri, küçükse geri gidilir. Hoca şu an basit bir çözüm aradıklarını, buna da sıra geleceğini söyledi.
- **Aranandan büyük bir elemana gelince döngüden çıkmak:** Hocanın "en basiti" dediği çözüm budur.

4.1 Koşulun Yönü: İlk Denemeler

`while` kısmı değiştirilecekti. `I <= N` korunacak, ikinci koşul yeniden yazılacaktı. Bir öğrenci önce `X <= Dizi[I]`, sonra `X < Dizi[I]` önerdi. Döngüden sonra da `Dizi[I] = X` ise I yazılacak, değilse "Eleman bulunamadı" yazılacaktı.

`X <= Dizi[I]` ile 1 arandığında $1 \leq 1$ doğru olduğu için döngü devam eder, $I = 2$ olur ve eleman kaçırılır. Bu yüzden `X < Dizi[I]` denendi. 1 için $1 < 1$ yanlıştır, döngüden çıkılır ve eleman bulunur.

> **Ders sonrası not:** Hoca bu koşulla 8 ve 9 aramalarını doğrudan son gözdeki 8 ile karşılaştırarak izledi ve 8'in bulunduğunu söyledi. Oysa `X < Dizi[I]` koşulunda 8 aranırken ilk adımda $8 < 1$ yanlıştır, döngüden $I = 1$'de çıkılır ve 8 bulunamaz. Hoca büyük olasılıkla hızlı geçtiği için bu adımı atladı. Hata hemen ardından gelen örnekte ortaya çıktı.

Hoca sonra başka bir dizi verdi: `5 8 20 32 45`. 9 aranırken ilk kontrol $9 < 5$ olur. Bu yanlış olduğu için döngüye hiç girilmez. Yani 8'e bakılmadan arama biter. Burada bir terslik vardır. Öğrencilerle konuşularak koşulun yönü değiştirildi: Döngü, aranan eleman baktığımız elemandan büyük olduğu sürece devam etmelidir.

```
I <= N ve X > Dizi[I] olduğu sürece:
  I = I + 1.
```

4.2 Sınır Sorunu: Dizinin Dışına Erişim

Hoca bu koşulu ilk dizideki 9 aramasıyla izledi ($N = 7$):

1. $I = 7$ iken $7 \leq 7$ doğru, $9 > 8$ doğrudur. Döngüye girilir.
2. $I = 8$ olur. $8 \leq 7$ yanlıştır, döngüden çıkılır.
3. Aşağıda `X = Dizi[I]` sorulur, yani `Dizi[8]`'e bakılır.

Sorun buradadır. `Dizi[8]` diziye ait olmayan bir gözdür.

Çözüm için farklı öneriler geldi:

- **Alttaki `if`'ten önce `I <= N` kontrolü eklemek:** Bir öğrenci bunu önerdi. Hoca başka bir yol olup olmadığını sordu.
- **`if` içinde `Dizi[I]` yerine `Dizi[I-1]` yazmak:** Hoca bunun 8 aramasında çalışmayacağını gösterdi. Bu yöntem 9'u doğru sonuçlandırır, ama döngünün içinde bulunan 8 gibi bir durumu bozar.
- **`if`'ten önce I'yı bir azaltmak:** Önceki öneriyle aynıdır. Döngüden hangi koşulla çıkıldığı bilinmez.
- **Koşulu `X > Dizi[1]` gibi iki parçalı yazmak:** Bu da hocaya göre fazla karışık oldu.

- **`while` koşulunu `I < N` yapmak:** Eşitlik kaldırılır. Hoca bu öneriyi denemeye değer buldu.

4.3 Doğru Çözüm: `I < N`

```
1. Başla.
2. N ve Dizi[N] oku.
3. X oku.
4. I = 1.
5. I < N ve X > Dizi[I] olduğu sürece:
     I = I + 1.
6. X = Dizi[I] ise I yaz.
   Değilse "Eleman bulunamadı" yaz.
7. Bitir.
```

Hoca üç durumu izledi:

- **1 aranıyor:** $1 > 1$ yanlıştır. Döngüye girilmez, `Dizi[1] = 1` olduğu için eleman bulunur.
- **8 aranıyor:** I 1'den başlar. $6 < 7$ denetiminden sonra $I = 7$ olur. $7 < 7$ yanlıştır, çıkılır. `Dizi[7] = 8` olduğu için eleman bulunur. Hocanın dikkat çektiği nokta şudur: Eleman döngünün içinde değil, çıktıktan sonra bulunmuştur.
- **9 aranıyor:** Yine $I = 7$'de çıkılır. `Dizi[7] = 8` ve $9 \neq 8$ olduğu için "Eleman bulunamadı" yazılır.

Soru: `while` koşulunu `I <= N` bırakıp alttaki `if`'i `X = Dizi[I]` yerine `I <= N` diye yazsak ne olur?

Cevap: O zaman döngüden eşitlik yüzünden mi çıkıldığı bilinmez. Koşul `X > Dizi[I]` olduğu için, döngüden aranan eleman baktığımızdan küçük olduğu için de çıkılmış olabilir, eşit olduğu için de.

4.4 İki Algoritmanın Farkı

Bir öğrenci önemli bir itiraz getirdi.

Soru: İkinci algoritmayı daha verimli olsun diye yazdık. Ama değişken sayısı aynı, yapılan işlemler aynı ve birinci de zaten çalışıyor. Neden yazdık?

Cevap: Soruyu başka bir öğrenci `5 8 20 32 45` dizisinde 9 aranması üzerinden cevapladı. Sıralı dizi algoritması 5'i ve 8'i kontrol eder, sonra 20'de durur. Geri kalan sayılar 9'dan büyük olduğu için onlara bakılmaz. Sırasız dizi algoritması ise dizinin sonuna kadar gider. Hoca iki çözümün görünüşte çok benzer olduğunu, ama birinin sonuna kadar gittiğini, diğerinin kendinden büyük elemanlar varsa kalanına bakmadığını söyledi. Bu ikinci çözüm yalnız sıralı dizi için geçerlidir.

> **Hocanın vurgusu:** Sınır değerleri çok önemlidir. Bu örnek, sınır değerlerinin önemini anlatmanın en basit yollarından biridir. Burada `I <= N` deseydik yanlış çalışırdı. Eşitliğin yönü de çok önemlidir, koşulu ters yazsaydık yine hatalı çalışırdı. Analiz yapmasaydık "oldu" deyip geçerdik.
>
> **Hocanın vurgusu:** Bunları sakın ezberlemeyin. Her zaman kağıt üzerinde bu işi elle nasıl yapacağınızı düşünün, sonra akış diyagramına çevirin.

4.5 İkili Arama

Hoca, bir öğrencinin önerdiği "ortasına bakıp ona göre arama" yaklaşımına geri döndü. Dizinin ortasına bakıp büyük ya da küçük olma durumuna göre ilerleyen bu yönteme binary search denir. Hoca bunu, vakit kalırsa dönem sonunda konuşacaklarını söyledi.

Bu noktada bir soru sordu: Şimdiye kadarki aramalar en kötü ihtimalle kaç adımda biter? Cevap, dizideki eleman sayısı kadar, yani N adımdır. Binary search ise bunu çok daha avantajlı bir adım sayısıyla sonlandırır.

> **Ek bilgi (derste anlatılmadı):** İkili aramada her karşılaştırma bakılacak aralığı büyük olasılıkla yarıya indirir. Bu yüzden en kötü durumdaki adım sayısı büyük olasılıkla $\log_2 N$ mertebesindedir. Yöntem dizinin sıralı olmasına dayanır.

5. Diziye Eleman Ekleme

Sonraki işlem ekleme (insert operation) oldu. Burada da dizinin sırasız ya da sıralı olması her şeyi değiştirir.

5.1 Sırasız Dizi

Elimizde sırasız bir dizi (unsorted array) varsa ve yeni bir eleman eklemek istiyorsak ne yaparız? Bir öğrenci önce elemanı `Dizi[I]`'ye yazmayı önerdi, hoca bunun olmayacağını belirtti. Öğrenci kendini düzeltip `Dizi[N+1]` dedi. Doğrusu budur.

Örneğin 10 geldiğinde dizinin sonuna ekleriz. Dizi sırasız olduğu için bozulacak bir sıra yoktur. Başa eklemek biraz maliyetli olur, çünkü bütün elemanları kaydırmak gerekir. En maliyetsiz yol, son elemandan sonraki göze, yani `Dizi[N+1]`'e yazmaktır.

5.2 Sıralı Dizi: Fikir

Dizi sıralıysa eleman, sırayı bozmayacak yere konmalıdır. Hoca 10 eklemeyi örnek verdi. Eklemeden sonra dizi `1 3 5 7 9 10 11` olur. Öğrencilere algoritmanın akış diyagramını kendilerinin çizmesi için süre verdi. Aradan sonra önce "kağıt üstünde ne yapmalıyız?" diye sordu.

Öğrencilerin önerisi şuydu: Dizi sıralı olduğu için eklenecek elemandan büyük ilk elemanı buluruz, sonra kaydırmaya başlarız. Hoca bunu `1 3 5 7 9 10 11` dizisine 6 ekleme örneğiyle somutlaştırdı: 1, 3 ve 5 yerinde kalır. 7, 9, 10 ve 11 birer göz sağa alınır. 6, 7'nin ilk bulunduğu yere yazılır. Sonunda dizinin boyutu bir artar.

Soru: Kaydırma işlemine nereden başlamalıyız?

Cevap: En son elemandan, yani önce 11 kaydırılır. Baştan başlarsak, örneğin 7'yi kaydırmaya çalışırsak 7'yi 9'un üstüne yazarız ve 9 kaybolur.

Kaydırma sondan başlar ve kendisinden büyük ilk bulunan elemanın olduğu yere kadar sürer. O eleman da kaydırıldıktan sonra bulunan noktaya yeni eleman yazılır.

5.3 Akış Diyagramı

Çözüm, öğrencilerin söylediği şekilde adım adım kuruldu. Yeri bulma kısmı, sıralı dizideki aramanın aynısıdır. Koşulda `I <= N` kullanıldı.

```
1. Başla.
2. N ve Dizi[N] oku.
3. Eklenecek elemanı X olarak oku.
4. I = 1.
5. I <= N ve X > Dizi[I] olduğu sürece:
     I = I + 1.
6. J ← N || -1 || I için (J, N'den I'ya birer azalarak):
     Dizi[J+1] = Dizi[J].
7. Dizi[I] = X.
8. N = N + 1.
9. Bitir.
```

8. adım öğrencinin sorusu üzerine sonradan eklendi (bkz. 5.7).

Bir öğrenci kendi çözümünde döngüden sonra `I <= N` kontrolü yaptığını, böylece son elemandan sonra eklerken kaydırmaya girmediğini söyledi. Hoca önce kontrolsüz hâli yazdırdı ve kontrolün gerekip gerekmediğini izleyerek görmek istedi.

5.4 Örnek: 6 Eklemek

Verilen: `1 3 5 7 9 10 11`, $N = 7$, $X = 6$.

Önce yer bulunur. $6 > 1$, $6 > 3$ ve $6 > 5$ doğru olduğu için devam edilir. $6 > 7$ yanlıştır, döngüden çıkılır. Sonuç $I = 4$'tür.

Sonra kaydırma yapılır. J, 7'den başlar ve 4'e kadar birer azalır:

J	Atama	Taşınan değer	Dizi (ara adım eklendi)
7	`Dizi[8] = Dizi[7]`	11	`1 3 5 7 9 10 11 11`
6	`Dizi[7] = Dizi[6]`	10	`1 3 5 7 9 10 10 11`
5	`Dizi[6] = Dizi[5]`	9	`1 3 5 7 9 9 10 11`
4	`Dizi[5] = Dizi[4]`	7	`1 3 5 7 7 9 10 11`

En son `Dizi[4] = 6` yazılır ve dizi `1 3 5 6 7 9 10 11` olur. Hoca algoritmanın bu durumda sorunsuz çalıştığını söyledi.

5.5 Uç Durumlar

Hoca, kontrol gerekip gerekmediğini anlamak için iki uç durumu izledi.

En büyük elemanı eklemek (12): 12, 11 dahil bütün elemanlardan büyüktür. Döngüde $I = 8$ olur, $8 \leq 7$ yanlıştır ve çıkılır. For döngüsünde $J = 7$'den başlanır, ama $7 \geq 8$ yanlış olduğu için döngüye girilmez. Yalnız `Dizi[8] = 12` yazılır. Herhangi bir kaydırma yapılmaz ve sorun çıkmaz. Yani önerilen ek kontrole gerek yoktur, for döngüsü bu kontrolü zaten yapar.

En küçük elemanı eklemek (0): $1 \leq 7$ doğru, $0 > 1$ yanlıştır. Döngüden $I = 1$ ile çıkılır. For döngüsü $J = 7$'den 1'e kadar gider: 7. göz 8.'ye, 6. göz 7.'ye, ..., 1. göz 2.'ye kopyalanır. Sonra `Dizi[1] = 0` yazılır. Burada da sorun yoktur.

> **Ders sonrası not:** Hoca 0 eklemenin sonucunu okurken diziyi "1, 3, 5, 7, 9, 10, 11, 12" diye saydı, ama kaydırmayı $J = 7$'den başlattı. Önceki örnekteki 12 büyük olasılıkla ekranda kalmıştı. `1 3 5 7 9 10 11` dizisine ($N = 7$) 0 eklenince sonuç `0 1 3 5 7 9 10 11` olur. 12 de eklenmiş olsaydı $N = 8$ olmalı ve kaydırma $J = 8$'den başlamalıydı, aksi hâlde `Dizi[8] = Dizi[7]` ataması 12'nin üzerine yazardı. 5.7'deki soru tam bu noktayla ilgilidir.

5.6 Tekrar Eden Elemanlar

Soru: Dizi sıralı, ama tekrar eden elemanlar olabilir. `X > Dizi[I]` yerine `X >= Dizi[I]` yazsaydık bir fark olur muydu?

Cevap: Hoca diziyi `1 3 5 5 7 9 10 11 11` olarak yazdı. 6 eklenirken bir fark yoktur, çünkü 6 bütün 5'lerden büyüktür. 5 eklenirken ise fark vardır.

- **`X > Dizi[I]` ile:** $5 > 1$ ve $5 > 3$ doğru, $5 > 5$ yanlıştır. Döngü ilk 5'te ($I = 3$) durur. Kaydırılan elemanlar 11, 11, 10, 9, 7, 5, 5 olur, yani 7 kaydırma yapılır (ara adım eklendi).
- **`X >= Dizi[I]` ile:** Döngü bütün 5'leri geçer, 7'de ($I = 5$) durur. Kaydırılan elemanlar 11, 11, 10, 9, 7 olur, yani 5 kaydırma yapılır (ara adım eklendi).

İki durumda da sonuç `1 3 5 5 5 7 9 10 11 11` olur. Soruyu soran öğrenci, döngünün duracağı yerin tekrar eden elemanların sonu olduğunu, böylece kaydırılacak eleman sayısının azaldığını açıkladı. Hoca bunun "ayrıntı gibi görünse de önemli" olduğunu söyledi: Tekrar eden eleman sayısı arttıkça onların kaydırılma olasılığı artar. Büyük eşittir kullanmak bizi bu kaydırmalardan kurtarır, çünkü her kaydırma bir maliyettir.

5.7 Dizinin Boyutunu Güncellemek

Bir öğrenci eklemede eksik kalan bir noktayı sordu.

Soru: Bir eleman ekledik, ama eleman sayısını algoritmanın neresinde arttırdık?

Sınıfta farklı cevaplar geldi. Bir öğrenci akış diyagramında boyut tanımlanmadığı için buna gerek olmadığını, bir diğeri J, N'den başladığı ve $J+1$'e yazıldığı için boyutun zaten arttığını söyledi.

Cevap: Hoca ikisine de karşı çıktı. Dizinin boyutunu temsil eden değişken N'dir. N şu an 7'dir ve bir eleman ekledik, öyleyse 8 olmalıdır. $N+1$'inci göze bir şey yazmakta sorun yoktur. Ama bir sonraki adımda dizi üzerinde bir iş yapılacaksa dizinin boyutu hâlâ N görünür. Bunun C'de kodlamakla ya da akış diyagramı çizmekle ilgisi yoktur.

Soru: Akış diyagramında kullanıcıdan aldığımız değişkenleri değiştirmemeyi tercih ettiğimizi söylemiştik. Yeni bir değişken mi açmalıyız?

Cevap: Burada dizinin eleman sayısı gerçekten değişiyor ve bu sayıyı temsil eden değişken N. Bu yüzden değeri değiştirilmelidir.

Nereye yazılmalı? Bir öğrenci `N = N + 1`'i `while`'dan hemen sonra koymayı önerdi. Hoca önce bunun da olabileceğini söyledi, sonra sorunun ne olacağını sordu. Başka bir öğrenci sorunu gösterdi: N 8 olursa for döngüsü $J = 8$'den başlar ve `Dizi[9] = Dizi[8]` ataması yapılır. Hoca bu tespiti doğruladı. Doğru yer for döngüsünden çıktıktan sonrasıdır. Böylece akış diyagramının 8. adımı eklendi.

Soru: N'yi arttırmadan for döngüsüne girdiğimizde `Dizi[N+1]`'e yazıyoruz. Bu göz bizde tanımlı değil, değil mi?

Cevap: Burada iki konu birbirine karışıyor. Birincisi, N'yi sonradan kullanma olasılığımız olduğu için arttırdık. İkincisi, dizinin boyutunu 100, 200, 300 diye tanımladığımız ve kullanıcı bundan düşük bir değer verdiği için $N + 1$'inci gözü zaten kullanabiliyoruz. Kullanıcıdan aldığımız sayıya göre yer ayırsaydık, bellekte yeniden yer ayırarak dizinin boyutunu arttırmamız gerekirdi. Bu, C'de ve bir sonraki sene öğrenilecek.

Hoca bir de şunu ekledi: Burada tek bir eleman ekledik. Başka bir dizinin elemanları eklenecek olsa bu işlem o dizideki eleman sayısı kadar tekrarlanırdı. Bunu daha akıllıca yapmanın taktikleri de vardır.

5.8 Diğer Sorular

Soru: `N, Dizi[N]` kısaltmasıyla okuma yapmaya bundan sonra da devam edebilir miyiz?

Cevap: Hoca bunun için erken olduğunu söyledi. Daha sonra, öğrencinin çiziminde `Dizi[I]` yazıldığını görünce de, doğrusunun `N` ve `Dizi[N]` yazmak olduğunu hatırlattı. Kısaltmayı süre kısıtlı olduğu için kabul ettiğini, aksi söylenmedikçe okuma biçiminin değişmediğini belirtti.

Soru: Sıralı dizi harflerden oluşsaydı, örneğin `A B C F G`, önce ASCII farklarını bulup o farklara göre mi sıralanır?

Cevap: Hayır, buna gerek yoktur. E harfini eklemek için doğrudan `X > Dizi[I]` demeye devam edebiliriz.

> **Ek bilgi (derste anlatılmadı):** C'de karakterler büyük olasılıkla sayısal kodları üzerinden karşılaştırılır. ASCII'de büyük harflerin kodları alfabe sırasıyla arttığı için `X > Dizi[I]` karşılaştırması büyük olasılıkla ek bir hesap gerektirmez.

5.9 Kaydırmanın Maliyeti

Hoca sorular üzerine kaydırma işleminin maliyetini tartıştı: Toplama da karşılaştırma da yapmıyoruz, öyleyse bilgisayardaki maliyet nedir?

Cevap, belleğe erişimdir. Kaydırmada hem bellekten okuma hem belleğe yazma vardır. Okuma biraz daha hızlı olsa da ikisi de CPU'dan çıkıp belleğe gitmek demektir. CPU ile bellek arasındaki bus bağlantısı, CPU'nun kendi içindeki veri aktarımından her zaman daha yavaş çalışır. Veriler cache'te saklanmıyor ve her seferinde belleğe gidip geliniyorsa, diziye erişmek de kontrol etmek ve dört işlem yapmak kadar maliyetlidir.

> **Hocanın vurgusu:** Atama işlemi sanki çok basitmiş, bir işlem yapılmıyormuş gibi görünür. Ama dizinin her bir gözüne erişmek bir maliyet demektir. Bunu unutmayın. Bu yüzden ardışık dizilerde (sequential array) ekleme maliyetlidir.

Hoca buna alternatif olarak linkli listeyi andı. Veriler belleğin farklı bölgelerinde saklanır ve birbirine bağlanır, bağlanan noktaların adresleri tutulur. Bir eleman geldiğinde kaydırma yapmadan ilgili iki elemanın arasına yerleştirilir ve diğer elemanlara dokunulmaz. Bu yapı önümüzdeki dönem, en geç gelecek sene ayrıntılı görülecek.

6. Diziden Eleman Silme

6.1 Sırasız Dizi

Hoca silme işlemini (delete operation) sırasız bir diziyle başlattı: `1 4 5 3 2 8 9`. 2'yi silmek istiyoruz, sonuç `1 4 5 3 8 9` olmalıdır.

Bir öğrencinin önerisi: 2'nin üzerine 8, 8'in üzerine 9 yazılır, sonda kalan 9 kaldırılır. Bu kez kaydırma sağa değil sola doğrudur. Eklemenin tersine, N'den başlanmaz, N'ye doğru gidilir.

Kaydırmanın nereden başladığı konusunda küçük bir düzeltme yapıldı. Bir öğrenci "silmek istediğimiz değerden başlarız" dedi, başka bir öğrenci silinecek elemanın yanındakinden başlandığını belirtti. Hoca ikincisinin ifadesini doğru buldu: Silinecek elemanın yanındaki elemanı alıp üzerine yazarak ilerleriz.

Kaydırma bitince dizinin son gözünde eski değer kalır. Hoca oraya 0 ya da -1 gibi bir değer yazılabileceğini, ama N'nin mutlaka bir azaltılması gerektiğini söyledi.

6.2 Sıralı Dizi

Sıralı dizi `1 3 5 7 8 9` için 7 silinince sonuç `1 3 5 8 9` olur. Fark nerededir? Bir öğrenci bunu doğru gösterdi: Silmek istediğimiz eleman dizide yoksa ve baktığımız eleman ondan büyükse durabiliriz. Yani fark, silmeye başlanacak noktayı arama kısmındadır:

- Sırasız dizide (unsorted array) bütün diziyi gezmek gerekir.
- Sıralı dizide aranan elemandan büyük bir elemana gelindiği an arama durdurulur.

Hoca öğrencilerden sıralı dizide verilen bir elemanı silen ve diziyi yeniden düzenleyen algoritmanın akış diyagramını istedi. Aradan sonra bir öğrencinin çizimini ekranda açıp onun üzerinden ilerledi.

> **Hocanın vurgusu:** Süre verilen zamanları iyi değerlendirin. Bu yöntem kendi başınıza kaldığınızda bir çözümü ne kadar ilerletebildiğinizi gösterir. Bu süreleri yalnız fikir sahibi olmak için değil, çözümü kendiniz kurmak için kullanın.

6.3 Öğrencinin Çizimi

Çizimde akış şöyleydi:

```
1. N ve Dizi[N] oku.
2. Silinecek elemanı X olarak oku.
3. I = 1.
4. I <= N ve X > Dizi[I] olduğu sürece:
     I = I + 1.
5. J ← I || 1 || N-1 için:
     Dizi[J] = Dizi[J+1].
6. N = N - 1.
7. Bitir.
```

Hoca 7 silmeyi izledi. $7 > 1$, $7 > 3$ ve $7 > 5$ doğru, $7 > 7$ yanlıştır. $I = 4$'te çıkılır. For döngüsünde J, 4'ten $N - 1 = 5$'e gider. `Dizi[4] = Dizi[5]` ile 8, 7'nin yerine geçer. `Dizi[5] = Dizi[6]` ile 9, 8'in yerine geçer. N 5 olur.

Soru: Son değeri silmeyi unutmadık mı?

Cevap: 8, 7'nin üzerine, 9, 8'in üzerine yazıldı ve boyut bir azaltıldı. Bu yeterlidir.

Soru: For döngüsü $N - 1$'e kadar değil, N'ye kadar gitmemeli mi? Yoksa 9 geri atanmaz.

Cevap: Atama `Dizi[J] = Dizi[J+1]` şeklinde olduğu için $N-1$ doğru sınırdır. $J = N-1$ iken zaten `Dizi[N]` okunur. N'ye kadar gidilseydi `Dizi[N+1]`'e ulaşılırdı. Atamalar farklı yazılırsa sınır değerleri de değişir.

> **Hocanın vurgusu:** J'nin I'dan başladığına dikkat edin. Bu çok önemlidir ve birçok problemde karşınıza çıkacak. Bir döngüde bulduğunuz noktayı başka bir döngünün başlangıç değeri olarak kullanabilirsiniz.

6.4 Eleman Dizide Yoksa

Hoca baştan söz etmediği bir noktayı açtı: Silme işleminde aradığımız eleman dizide olmayabilir.

`1 3 5 7 8 9` dizisinden 6 silinmek istensin. $6 > 1$, $6 > 3$ ve $6 > 5$ doğru, $6 > 7$ yanlıştır ve $I = 4$'te çıkılır. Çizimdeki akışta bu noktada doğrudan for döngüsüne girilir ve kaydırma başlar. Oysa 6 bulunmamıştır, çizim 7'yi siler.

Öğrencilerden iki öneri geldi: bulunup bulunmadığını gösteren yeni bir değişken eklemek ya da ek bir değişken kullanmadan `X = Dizi[I]` kontrolü yapmak. İkincisi seçildi. Bu kontrol `while` ile `for` arasına konur. 6 için $6 = 7$ yanlıştır ve for döngüsüne girilmez.

6.5 `I < N` Düzeltmesi

Kontrol eklenince yeni bir sınır sorunu çıkar. 11 silinmek istensin: 11 dizideki bütün elemanlardan büyüktür. $I = 6$ iken $11 > 9$ doğrudur, $I = 7$ olur ve çıkılır. Sonra `Dizi[7]` ile karşılaştırma yapılır, ama bu göz diziye ait değildir.

Hoca çözümü öğrencilere sordu: `while` koşulundaki eşittir kaldırılsa olur mu? Bir öğrenci kendisinin de böyle yaptığını, döngüden çıkınca `Dizi[I] = X` kontrolünü koyduğunu söyledi. Bu, sıralı dizide aramada görülen düzeltmenin aynısıdır. Koşul `I < N` olduğunda:

- Silinecek eleman son elemana eşitse bulunur.
- Son elemandan da büyükse eşit olmadığı için bulunamaz, false koluna girilir. Bu kolda for döngüsü olmadığı için kaydırma yapılmaz, "bulunamadı" yazılır.

Son akış diyagramı şöyledir:

```
1. Başla.
2. N ve Dizi[N] oku.
3. X oku.
4. I = 1.
5. I < N ve X > Dizi[I] olduğu sürece:
     I = I + 1.
6. X = Dizi[I] ise:
     J ← I || 1 || N-1 için: Dizi[J] = Dizi[J+1].
     N = N - 1.
   Değilse:
     "Eleman bulunamadı" yaz.
7. Bitir.
```

6.6 Sorular

Soru: `while` koşulunu `X ≠ Dizi[I]` yazsaydık olmaz mıydı?

Cevap: Yanlış çalışmazdı. Ama aranan elemandan büyük elemanlar da gereksiz yere kontrol edilirdi. İş yükü artacağı için tercih etmiyoruz.

Soru: Kontrolü `while` ile `for` arasına, koşulu `I < N` yaptıktan sonra koyacağız, değil mi?

Cevap: Evet.

Soru: Tekrar eden elemanların hepsini silmek istersek, örneğin `1 3 5 5 5 7`?

Cevap: Öğrenci önce bir `while` döngüsüyle kaç tane 5 olduğunu saydığını, sonra atamayı 3 göz geriye yapacak şekilde değiştirdiğini anlattı. Hoca bunu şöyle özetledi: Eleman kaç kez tekrar ediyorsa o sayı bulunur. Kaydırmaya başlanacak eleman tekrarların bittiği yerdedir. O noktadan itibaren elemanlar tekrar sayısı kadar geriye kaydırılır. Üç tane 5 varsa 7, üç 5'in başladığı yere alınır. Hoca bunu ayrıntısıyla çizmedi ve kendi kendine çözülmesini tavsiye etti.

7. Güncelleme ve Günlük Hayattan Örnekler

Hoca güncellemeyi ayrıca çizmedi. Güncelleme aramanın birebir aynısıdır: Bulunan yerde yalnız değer değiştirilir. Böylece temel işlemler tamamlanmış oldu.

Bu problemlerin varyasyonları vardır: birden fazla eleman silmek, tekrar eden elemanları silmek, birden fazla eleman eklemek. Hoca, soru çözmek isteyenlerin bu varyasyonları çoğaltıp günlük hayattan problemler üzerinde çalışabileceğini söyledi. Bu tür uğraşların algoritma geliştirme yeteneğini arttırdığını da ekledi.

Bu işlemlerin nerede kullanıldığını sorduğunda şu örnekler konuşuldu:

- **Silme:** Yurttan bir öğrenci ayrıldığında yurttaki bilgileri silinir. Bir sınav iptal olduğunda notlar geri çekilir. Alışveriş yaparken sepetten ürün çıkarılır.
- **Güncelleme:** Bir not yanlış toplandıysa düzeltilir. Sepetteki ürün sayısı birden ikiye çıkarılır.
- **Ekleme:** Sonuçlar açıklandığında notlar eklenir. Alışveriş sepetine ürün eklenir.

Hoca bunların hepsinin günlük hayatımızda fazlasıyla yapılan işler olduğunu ve birçok şeyin bunların etrafında döndüğünü söyledi.

8. Kaydırma Maliyetinden Kaçınmak

Soru: Bir milyon verinin olduğu yerde bir tane elemanı silmek istesek çok fazla işlem yapıyormuşuz gibi geliyor. Bu yükü engellemenin bir yolu var mı?

Cevap: Tabii ki var. Silerken diziyi kaydırmak çok maliyetli olduğu için farklı yöntemler kullanılır, verinin saklanma şekli de değişir. Hoca şu yolları saydı.

Linkli liste: Bu yapıyla silme ve ekleme çok daha hızlı yapılır, çünkü kaydırmaya gerek kalmaz.

İşaretleme: Silinen göze 0 ya da -1 gibi bir değer yazılır, yani "burası silindi" diye işaretlenir. Bir öğrenci önce boşluk değeri önermişti. Hoca bunun çok sık kullanılan bir yöntem olduğunu söyledi. Kaydırma yapılmadan veri silinmiş olur. Dezavantajı, o gözün hâlâ işgal ediliyor olmasıdır. Dizi küçülmemiştir, yani space complexity hâlâ yüksektir.

Silinen gözleri ayrı bir dizide tutmak: Bir öğrenci okuma sırasında silinecek değere gelince `if` ile atlamayı önerdi. Hoca bunun olmayacağını söyledi, çünkü bütün gözler için bu kontrolü yapmak gerekir ve bu ekstra bir sorun yaratır. İşaretleme yapıldıysa zaten o göze bakınca değer olup olmadığı anlaşılır. Bir varyasyon olarak silinen gözlerin yerleri başka bir dizide tutulabilir ve okuma bu yerler dikkate alınmadan yapılabilir.

Boşalan yerleri yeniden kullanmak: Bir öğrenci, dizi sırasızsa ve yine bir eleman eklenecekse silinen yere doğrudan eklenebileceğini söyledi. Hoca bunun doğru bir yaklaşım olduğunu belirtti ve bilgisayarın belleğiyle ilişkilendirdi. Bir programın işi bittiğinde kullandığı alanlar boşa çıkar. O alanlarda eski bilgiler hâlâ yüklüdür, ama alanların boşa çıktığı bilgisi işletim sisteminde tutulur. Başka bir program bellekte yer istediğinde işletim sistemi bu adreslere sahip olduğu için o alanları yeniden kullanıma açar. Aynı şey dizi üzerinde de uygulanabilir. Hocaya göre ikincil bellekteki çalışma prensibi de budur.

Hoca buna flash disk biçimlendirmeden örnek verdi. "Hard reset"te bütün bellek yerlerindeki bilgilerin üzeri silinir, her birine sıfır bastığınızı varsayın. Bu daha uzun sürer. "Soft reset"te ise oradaki değerler kalır, yalnız alanın kullanıma açıldığı bilgisi işletim sistemi tarafından bir yerde saklanır.

> **Ders sonrası not:** Hoca flash disk biçimlendirme için "hard reset" ve "soft reset" terimlerini kullandı. Bu iki yöntemin yaygın adı büyük olasılıkla tam biçimlendirme (full format) ve hızlı biçimlendirmedir (quick format). Anlatılan fark bununla uyumludur.

Hoca bu konuyu bitirirken algoritmaların kodlarını yazmayı şiddetle tavsiye etti: Çalıştırıp sonuçları görmek ve farklı durumlar için sonuç üretmek gerekir. Bu dersin amacı C öğretmek değil, yalnız akış diyagramlarını programa çevirmektir. Ama ikinci dönemde ve özellikle ikinci sınıfın birinci döneminde C ile ilgili çok ciddi bilgiler öğrenilecektir.

9. Sıralama Problemi ve Bubble Sort

9.1 Sıralamanın Yeri

Dizinin sıralı ya da sırasız olması her şeyi değiştirir. Sıralı dizilerde çalışmak avantaj sağlar. Ama "dizi sırasızsa önce sıralayayım, sonra işimi yapayım" demek de az maliyetli bir iş değildir, diziyi sıralamanın önemli bir maliyeti vardır.

Sıralama, diziler üzerinde çok sık yapılan başka bir iştir. Hoca önce bubble sort ile başlayacaklarını, sonra selection sort'a geçeceklerini söyledi. İkinci sınıfın sonunda, veri yapıları dersinden sonra yaklaşık 10 farklı sıralama algoritması öğrenilmiş olacaktır. Bunların her birinin stili farklıdır, ama hepsi sayıların birbirleriyle karşılaştırılmasına dayanır. Karşılaştırmaya dayanmayan birkaç yöntem de vardır. Bunların time complexity'si daha iyidir, yani işlem sayıları daha azdır, ama başka dezavantajları vardır.

9.2 İlk Tur

Bubble sort'un mantığı, yan yana olan iki değerin birbiriyle karşılaştırılmasına dayanır. Hoca bunu iki kez tekrarladı. Örnek dizi `5 1 3 2 4 7 8 6`, amaç küçükten büyüğe sıralamaktır.

İlk tur şöyle ilerler:

Karşılaştırma	Sonuç	Dizi
5 ile 1	yer değişir	1 5 3 2 4 7 8 6
5 ile 3	yer değişir	1 3 5 2 4 7 8 6
5 ile 2	yer değişir	1 3 2 5 4 7 8 6
5 ile 4	yer değişir	1 3 2 4 5 7 8 6
5 ile 7	yerinde kalır	1 3 2 4 5 7 8 6
7 ile 8	yerinde kalır	1 3 2 4 5 7 8 6
8 ile 6	yer değişir	1 3 2 4 5 7 6 8

İlk turda en büyük eleman en sona gelmiştir. Hoca bunu şöyle genelledi: Dizinin başında 5 yerine 9 olsaydı, sonda 9 olurdu. Birinci turda en büyük eleman, ikinci turda ikinci en büyük eleman, üçüncü turda üçüncü en büyük eleman yerine götürülür.

9.3 Karşılaştırmaya Nereden Başlanır?

Bir öğrenci ikinci turda ikinci değerden başlanabileceğini önerdi. Bu örnekte 1 başa geldiği için sorun görünmez. Başka bir öğrenci sorunu fark etti: 1 başka bir yerde olsaydı yine başa geçer miydi? Hoca bunun üzerine 1'in en sonda, 6'nın yerinde olduğunu varsaydı. O zaman 1'in başa taşınması gerekir. İlk ikisi karşılaştırılmazsa 1 başa taşınamaz. Demek ki karşılaştırmaya her turda en baştan başlanır.

9.4 Kaç Döngü Gerekir?

İki iç içe döngü gerekir:

- **İç döngü:** Baştan sona tarayarak en büyük elemanı en sona iter.
- **Dış döngü:** Bu işi bütün elemanları sıralayabilmek için tekrarlar.

İç döngü 1'den başlar. N'ye kadar değil, $N-1$'e kadar gider, çünkü son karşılaştırma `Dizi[N-1]` ile `Dizi[N]` arasındadır.

9.5 Yer Değiştirme (Swap)

Soldaki eleman büyükse iki elemanın yeri değiştirilir. Hoca sorunu şöyle koydu: 1'i 5'in üzerine yazarsam 5 kaybolur, 5'i 1'in üzerine yazarsam 1 kaybolur. Bu klasik bir yer değiştirme işlemidir.

Bir öğrenci `Dizi[J]`'yi A'ya, `Dizi[J+1]`'i B'ye alıp sonra ters atamayı önerdi. Hoca iki değişkene gerek olmadığını, tek bir geçici alanın yeterli olduğunu gösterdi: 5 geçici alana alınır, 1, 5'in üzerine yazılır, geçici alandaki değer de eski 1'in yerine yazılır.

```
TMP = Dizi[J]
Dizi[J] = Dizi[J+1]
Dizi[J+1] = TMP
```

9.6 Akış Diyagramı

```
1. Başla.
2. N ve Dizi[N] oku.
3. I ← 1 || 1 || N-1 için:
     J ← 1 || 1 || N-1 için:
       Dizi[J] > Dizi[J+1] ise:
         TMP = Dizi[J].
         Dizi[J] = Dizi[J+1].
         Dizi[J+1] = TMP.
4. Bitir.
```

Dış döngü de $N-1$ kez döner. Hoca 8 elemanlı dizide bunu sayarak gösterdi: 1. turda 8, 2. turda 7, 3. turda 6, 4. turda 5, 5. turda 4, 6. turda 3, 7. turda 2 yerine oturur. 8. tura gerek yoktur.

Soru: Her turda en büyük değeri sona götürüyoruz. O zaman iç döngüde gittiğimiz son yeri birer azaltamaz mıyız?

Cevap: Hoca bunun çok mantıklı olduğunu söyledi ve algoritmanın neresinin değişeceğini sordu. Bir öğrenci `if`'in sonunda N'yi azaltmayı önerdi. Hoca N'yi değiştirmek istemedi. Başka bir öğrenci iç `for`'da `N-1` yerine `N-I` yazmayı önerdi. Doğrusu budur.

```
I ← 1 || 1 || N-1 için:
  J ← 1 || 1 || N-I için:
    Dizi[J] > Dizi[J+1] ise TMP ile yer değiştir.
```

Son eleman ilk turda yerine oturur, ikinci turda sondan ikinci eleman oturur. Yerine oturan elemanlar boşu boşuna karşılaştırılmaz. Hoca bu değişikliğin işi daha az işlemde tamamlamayı sağladığını söyledi.

> **Ek bilgi (derste anlatılmadı):** Dersteki dizi izlendiğinde 2. turun sonunda `1 2 3 4 5 6 7 8` elde edilir ve kalan 5 turda hiç yer değişmez. 8 eleman için toplam karşılaştırma `N-1` sınırıyla $7 \times 7 = 49$, `N-I` sınırıyla $7+6+\cdots+1 = 28$ olur. Hocanın sonraki derse bıraktığı "daha akıllıca" sorusu büyük olasılıkla bu gereksiz turlarla ilgilidir.

Dersin Toparlaması

Bu derste dizilerdeki dört temel işlem, sıralı ve sırasız diziler için ayrı ayrı ele alındı. Sırasız dizide arama `I <= N ve Dizi[I] ≠ X` koşuluyla bütün diziyi gezer. Sıralı dizide `I < N ve X > Dizi[I]` koşulu, aranandan büyük elemana gelince durur ve dizinin dışına erişimi önler. Sıralı diziye eklemede önce yer bulunur, sonra elemanlar sondan başlanarak sağa kaydırılır, eleman yazılır ve for döngüsünden sonra N bir arttırılır. Sıralı diziden silmede önce yer bulunur, elemanın gerçekten orada olduğu `X = Dizi[I]` ile kontrol edilir, sonra elemanlar sola kaydırılır ve N bir azaltılır. Güncelleme aramanın aynısıdır. Bu örneklerin her birinde küçük görünen sınır ve eşitlik seçimlerinin algoritmanın doğruluğunu ve işlem sayısını nasıl değiştirdiği görüldü.

Son olarak sıralama problemine geçildi ve yan yana elemanları karşılaştırıp her turda en büyüğü sona taşıyan bubble sort kuruldu. Bir sonraki derste bubble sort tekrar edilecek, iyileştirilmiş bir versiyonu konuşulacak ve selection sort'a geçilecek. Hoca öğrencilerden bubble sort'u daha akıllıca hâle getirmenin yolunu düşünmelerini istedi.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.